Universidade Federal do Maranhão
Centro de Ciências Exatas e Tecnologia — Departamento de Engenharia da Computação
Disciplina: Sistemas Distribuídos — Docente: Prof. Dr. Luiz Henrique Neves Rodrigues

Relatório Técnico de Evolução Arquitetural

Transição do Middleware RabbitMQ para Barramento Híbrido ESB + ORB, Replicação Fanout e Tolerância Sistemática a Falhas (Marcos 3 e 4)

Discentes: Eduardo dos Santos Oliveira, Fernando da Silva Costa, Herick Vinicius Pinheiro Da Conceição, Samara Santos Viegas e Vyktor Gabryel Costa de Souza

---

1 Introdução e Contexto do Problema

Em ambientes hospitalares modernos, particularmente em Unidades de Terapia Intensiva (UTI), os sinais vitais de pacientes (frequência cardíaca, saturação de oxigênio SpO2, pressão arterial e temperatura) são gerados em alta frequência por monitores multiparamétricos à beira do leito. Um sistema distribuído encarregado de processar esse fluxo de dados enfrenta desafios rigorosos:

1. **Missão Crítica**: A perda de um único pacote contendo um alerta de parada cardiorrespiratória ou de queda do leito pode comprometer vidas. O sistema deve operar sob garantia estrita de *Zero Data Loss*.
2. **Tolerância a Falhas Parciais**: Em um cluster distribuído, nós servidores podem falhar a qualquer momento devido a esgotamento de memória, sobrecarga de CPU ou falhas de rede. A infraestrutura deve continuar operando mesmo com a queda de um ou mais nós (**Marco 4**).
3. **Escalabilidade e Replicação Ativa**: Os dados devem ser replicados de forma consistente entre múltiplas instâncias para permitir leitura paralela e sobrevivência a desastres (**Marco 3**).

O presente documento detalha a transição arquitetural realizada no núcleo de processamento do sistema: a substituição do broker externo tradicional (**RabbitMQ**) por uma solução de **Middleware Corporativo Híbrido** (integrando **ESB** com pipeline VETRO e **ORB** com serviço de nomes CORBA CosNaming), associada a técnicas de resiliência (*Circuit Breakers*, *Retry* com recuo exponencial e *Dead-Letter Queues*).

2 Motivação da Substituição do RabbitMQ: Análise de Limitações Técnicas

2.1 Transmissão Agnóstica (Dumb Pipes) vs. Mediação Ativa (Smart Bus)

Inicialmente, o sistema utilizava o RabbitMQ operando via protocolo AMQP na porta 5672 para enfileirar as mensagens entre o cliente produtor e os servidores de persistência. Embora o RabbitMQ seja amplamente utilizado na indústria para mensageria assíncrona desacoplada, sua arquitetura baseia-se no princípio clássico de *Dumb Pipes and Smart Endpoints*: o broker atua estritamente como um canal passivo de transporte de bytes, delegando todo o tratamento semântico exclusivamente às extremidades.

Princípio Arquitetural:

Incompatibilidade de Brokers Tradicionais com Telemetria Crítica Em fluxos contínuos de sinais vitais, a transmissão passiva é insuficiente. O sistema demanda **mediação ativa**: o barramento necessita inspecionar o payload em trânsito para validar limites fisiológicos, normalizar dados heterogêneos para padrões interoperáveis (como HL7/FHIR) e aplicar roteamento baseado no conteúdo (*Content-Based Routing*).
Transferir essas responsabilidades para os nós finais gera duplicação de lógica e acoplamento desnecessário. Além disso, a dependência de um processo *daemon* monolítico externo introduz um Ponto

Único de Falha (*Single Point of Failure* - SPOF), onde a indisponibilidade do broker paralisaria toda a comunicação entre clientes e servidores de persistência.

2.2 Limitações Técnicas do RabbitMQ no Cenário Hospitalar

1. **Ponto Único de Falha (SPOF) Monolítico**: A operação dependia de um processo *daemon* externo de terceiros (`rabbitmq-server`) executando sobre a máquina virtual Erlang (BEAM). Caso o daemon sofresse pane ou a porta AMQP 5672 fosse bloqueada, todo o fluxo era interrompido.
2. **Agnosticismo Semântico de Carga Útil**: O RabbitMQ enfileira sequências opacas de bytes (*byte payloads*). Ele não valida conformidade com padrões hospitalares (como HL7 ou FHIR), não calcula escores de risco e não descarta dados corrompidos antes da entrega.
3. **Sobrecarga de Protocolo AMQP**: O handshake de conexão, confirmações por canal (*channel acks*) e negociação de prefetch criam latência desnecessária quando comparados a uma arquitetura orientada a serviços com despacho direto e transacional.
4. **Incompatibilidade com Objetos Distribuídos**: O RabbitMQ é um middleware puramente orientado a mensagens (MOM). Ele não oferece suporte nativo a invocação remota orientada a objetos (RPC / CORBA) ou resolução hierárquica de nomes de serviços.

3 A Nova Arquitetura: Middleware Híbrido ESB + ORB

Para superar as limitações do RabbitMQ, concebeu-se um middleware nativo completo que combina dois paradigmas clássicos e consolidados da computação distribuída:

Decisão de Engenharia:

Padrão Híbrido: Barramento de Serviços + Barramento de Objetos A nova solução integra nativamente:

- **Enterprise Service Bus (ESB)**: Responsável pelo desacoplamento de fluxos, validação semântica clínica e roteamento inteligente através do padrão **Pipeline VETRO**.
- **Object Request Broker (ORB - CORBA)**: Responsável pela interoperabilidade de componentes, transparência de localização e resolução dinâmica de instâncias através do padrão **CosNaming**.

3.1 O Enterprise Service Bus (ESB) e o Pipeline VETRO

O padrão **VETRO** é uma referência internacional em integração de sistemas corporativos (*Enterprise Integration Patterns*). No contexto deste projeto, cada dado de sensor clínico atravessa sequencialmente cinco etapas atômicas antes de alcançar os bancos de dados:

Fase	Conceito e Mecanismo Técnico	Papel no Domínio Hospitalar
V	**Validate (Validação Semântica)**: Verifica tipagem e se os valores fisiológicos estão dentro de intervalos humanamente possíveis. Registros anômalos são isolados.	FC $\in [30, 220]$ bpm, SpO2 $\in [50, 100]\%$, Temp $\in [30, 45]°C$, PA $\in [40, 260]$ mmHg.
E	**Enrich (Enriquecimento Clínico)**: Computa variáveis secundárias clínicas antes do armazenamento, agregando inteligência hospitalar ao dado bruto.	Cálculo determinístico do escore **NEWS2** e prioridade da **Triagem de Manchester**.
T	**Transform (Transformação e Normalização)**: Converte esquemas heterogêneos de sensores médicos para o modelo canônico interoperável **HL7 v2** / **FHIR JSON**.	Padronização de chaves, carimbos temporais ISO-8601 e identificador UUID unívoco.
R	**Route (Roteamento Baseado em Conteúdo)**: O roteador de conteúdo (*Content-Based Router*) inspeciona a triagem para definir a fila de despacho.	Emergência e Queda → Rota prioritária UTI; Casos estáveis → Rota geral.
O	**Operate (Operação Atômica & Fanout)**: Executa a replicação multi-nó (*fanout*), persistência nos bancos locais e notificação de telemetria.	Gravação síncrona nos SQLite locais com garantia de consistência e descarte de duplicatas.

Tabela 1: Detalhamento técnico das fases do Pipeline VETRO do ESB hospitalar.

Fundamentação Teórica:

Cálculo Determinístico do NEWS2 e Manchester O **NEWS2** (*National Early Warning Score 2*) atribui pontuações baseadas na magnitude do desvio fisiológico:

$$\text{Escore}_{\text{NEWS2}} = f(\Delta\text{FC}) + f(\Delta\text{SpO2}) + f(\Delta\text{Temp}) + f(\Delta\text{PA}) + \text{AlertaQueda} \tag{1}$$

Com base no escore resultante, o middleware aplica a categorização da Triagem de Manchester:

- **Vermelho (Emergência)**: Escore ≥ 7 ou ocorrência de queda súbita. Tempo de resposta imediato (0 min).
- **Laranja (Muito Urgente)**: Escore de 5 a 6. Atendimento em até 10 min.
- **Amarelo (Urgente)**: Escore de 3 a 4. Atendimento em até 60 min.
- **Verde (Pouco Urgente)**: Escore de 1 a 2. Atendimento em até 120 min.
- **Azul (Não Urgente)**: Escore 0. Atendimento eletivo em até 240 min.

3.2 O Object Request Broker (ORB) e o Serviço de Nomes CosNaming

Princípio Arquitetural:

Transparência de Localização vs. Acoplamento de Endereçamento Físico O endereçamento direto baseado em URLs fixas (`http://ip:porta`) introduz acoplamento espacial severo na arquitetura distribuída. Caso uma réplica servidora seja reiniciada em outra porta, transferida de nó computacional ou escalada horizontalmente, os nós emissores perdem a conectividade até que seus parâmetros de rede sejam manualmente reconfigurados.
O padrão **CORBA CosNaming** elimina esse acoplamento introduzindo um **Serviço de Nomes Federado**: ele desacopla o nome lógico da sua localização física em rede, assegurando *Location Transparency*. O cliente referencia apenas a árvore lógica de identificadores (ex.: `Hospital/UTI/Server01`), enquanto o mecanismo de resolução do ORB traduz dinamicamente essa chave em uma Referência Interoperável de Objeto (*Interoperable Object Reference* - IOR), viabilizando portabilidade e reconfiguração dinâmica transparente.

O módulo ORB implementado fornece:

- **Árvore Hierárquica de Nomes**: Contextos delimitados que estruturam o hospital em domínios lógicos:
  - `Hospital/Core/Telemetria` → Objeto servidor principal de ingestão;
  - `Hospital/UTI/Server01` → Réplica servidora alocada para leitos de terapia intensiva;
  - `Hospital/Enfermaria/Server02` → Réplica servidora alocada para leitos gerais;
  - `Hospital/Cardiologia/Server03` → Réplica servidora especializada em telemetria cardíaca.
- **Interoperable Object Reference (IOR)**: String padronizada codificando metadados de protocolo, endpoint e identificador de objeto (`IOR:orb+http://...`), assegurando transparência de localização (*Location Transparency*). Se um servidor migrar de porta ou máquina, apenas o binding no CosNaming é atualizado.

4 Replicação Ativa e Escalabilidade (Marco 3)

4.1 Modelo de Replicação Ativa Fanout com Zero Data Loss

Para cumprir o Marco 3, o sistema rejeitou o modelo de banco de dados centralizado compartilhado. Em sistemas distribuídos clássicos, um banco de dados central cria um funil de concorrência e um ponto único de falha fatal: se o banco cair, todos os servidores caem simultaneamente.

Decisão de Engenharia:

Arquitetura de Bases Isoladas com Replicação Ativa Cada servidor do cluster opera com sua própria base de dados SQLite local independente e dedicada:

- Servidor 01 (Porta 8001) → Base `server1.db`
- Servidor 02 (Porta 8002) → Base `server2.db`
- Servidor 03 (Porta 8003) → Base `server3.db`
- Servidor 04 (Porta 8004) → Base `server4.db`

Fundamentação Teórica:

Mecanismo de Despacho Fanout Atômico A operação *Operate* do ESB realiza um *Broadcast* estruturado para todas as réplicas ativas:

1. O middleware calcula o hash SHA-256 da mensagem: $\mathcal{H} = \text{SHA256}(\text{paciente_id} \parallel \text{timestamp} \parallel \text{sinais_vitais})$.
2. A mensagem é enviada concorrentemente para todos os servidores registrados no pool de replicação.
3. Cada servidor recebe o payload e executa uma inserção protegida por chave primária unívoca (`INSERT OR IGNORE`). Caso ocorra uma retransmissão de rede, a operação é **idempotente**: o registro duplicado é silenciosamente ignorado sem corromper métricas.
4. Mesmo que 2 dos 3 servidores caiam por pane física, o servidor sobrevivente retém 100% do histórico clínico intacto.

4.2 O Papel do Servidor 04 como Warm Standby Elástico

Uma réplica reserva tradicional costuma ser fria (*cold standby*, desligada) ou exigir procedimentos complexos de recuperação. No projeto, o Servidor 04 foi implementado no modo **Warm Standby**:

- **Em Espera**: Ele não recebe requisições diretas do cliente através do Balanceador de Carga (o pool ativo atende com Servidores 01, 02 e 03).
- **Sincronização Contínua**: Ele continua conectado ao barramento de replicação fanout, gravando todas as telemetrias em `server4.db` em tempo real.
- **Scale-Out Instantâneo**: Quando a taxa de requisições por segundo (TPS) satura ou um dos servidores principais cai, o Servidor 04 é promovido instantaneamente ao pool ativo com latência nula de aquecimento (*zero warm-up latency*), pois sua base de dados já está perfeitamente espelhada.

5 Tolerância a Falhas e Resiliência Distribuída (Marco 4)

O célebre aforismo de Leslie Lamport define com perfeição o desafio do Marco 4: *"Um sistema distribuído é aquele em que o crash de um computador do qual você nunca ouviu falar pode tornar o seu próprio computador inútil"*.

Para garantir que o sistema sobreviva a falhas parciais, quedas abruptas de servidores (*crash faults*) e lentidões de rede (*performance degradations*), foram implementados quatro padrões canônicos de resiliência:

5.1 1. O Padrão Circuit Breaker (Isolamento Ativo de Falhas)

Princípio Arquitetural:

Isolamento de Falhas e Mitigação de Efeitos em Cascata Em arquiteturas distribuídas com dependências síncronas ou concorrentes, a degradação de um único nó (provocada por esgotamento de memória, bloqueio de I/O em disco ou sobrecarga de CPU) propaga-se rapidamente por todo o cluster. Clientes continuam encaminhando requisições ao nó moribundo, retendo threads em espera, exaurindo pools de conexões TCP e elevando a latência média do sistema de forma descontrolada (*cascading failure*).

O padrão **Circuit Breaker** (formalizado por Michael Nygard) introduz um mecanismo de proteção ativo e automático. Ao constatar um número consecutivo de falhas que excede o limiar de tolerância ($\theta_{\text{fail}} = 5$), o circuito transiciona para o estado **OPEN**, interceptando novas requisições e retornando falha imediata (*fail-fast*). Essa contenção estanca o desperdício de recursos no balanceador e concede ao nó degradado uma janela temporal de alívio sem sobrecarga para que possa drenar filas pendentes e restabelecer sua integridade operacional.

O Circuit Breaker opera sob uma Máquina de Estados Finita com três estados:

Estado	Comportamento Operacional	Condição de Transição
CLOSED (Fechado)	O circuito permite passagem normal do tráfego. O nó é considerado saudável. Cada requisição bem-sucedida zera o contador de erros.	Se as falhas consecutivas ultrapassarem o limiar $\theta_{\text{fail}} = 5$, transiciona para **OPEN**.
OPEN (Aberto)	O circuito bloqueia imediatamente todas as requisições destinadas àquele nó. As chamadas falham instantaneamente (*fail-fast*) sem onerar a rede.	Após transcorrer o tempo de recuperação $T_{\text{timeout}} = 10$ s, transiciona para **HALF-OPEN**.
HALF-OPEN (Semiaberto)	O circuito permite a passagem de uma requisição de teste para avaliar se o servidor se recuperou da pane.	Se o teste tiver sucesso → **CLOSED**. Se o teste falhar → Retorna para **OPEN**.

Tabela 2: Estados e regras de transição do Circuit Breaker implementado por nó servidor.

5.2 2. Retry com Recuo Exponencial e Jitter

Quando uma requisição falha por instabilidade transitória de rede, reenviá-la imediatamente de forma repetida gera o fenômeno conhecido como *Thundering Herd Problem* (efeito manada), saturando de vez o

servidor que já estava com dificuldades.

A estratégia adotada calcula o intervalo de espera para a tentativa k através da seguinte formulação matemática:

$$T_{\text{espera}}(k) = \min\left(T_{\max},\, T_{\text{base}} \times 2^{k}\right) + \text{Uniforme}(0,\, \text{Jitter}) \tag{2}$$

Onde $T_{\text{base}} = 0.5$ s, $T_{\max} = 10.0$ s e o **Jitter** adiciona uma variação aleatória pseudo-estocástica. Essa aleatoriedade desincroniza os múltiplos clientes emissores, espalhando os picos de tráfego uniformemente no tempo.

5.3 3. Fila de Cartas Mortas (Dead-Letter Queue - DLQ)

Dados que apresentam erros irrecuperáveis (como corrupção semântica em que a frequência cardíaca é negativa ou o identificador do paciente é nulo) nunca devem ser descartados silenciosamente nem reenviados eternamente em loop.

O middleware redireciona esses eventos para a **DLQ**:

- O registro é gravado em área isolada de quarentena acompanhado do motivo do descarte (*error code*, carimbo de data/hora e identificador do nó rejeitador).
- A auditoria hospitalar pode inspecionar a DLQ a frio para determinar falhas nos sensores físicos dos leitos, sem comprometer a vazão do barramento principal.

5.4 4. Padrão Saga para Transações Distribuídas

Diferente de sistemas corporativos simples onde se aplica o protocolo pesado de *Two-Phase Commit* (2PC) — que bloqueia recursos e é altamente vulnerável a falhas de coordenador —, o sistema emprega o **Padrão Saga**:

- A persistência em múltiplos servidores é tratada como uma sequência de transações locais encadeadas.
- Se um nó rejeita a escrita por violação de integridade após outros nós terem confirmado, o orquestrador do middleware emite transações compensatórias para anular os efeitos locais, assegurando **Consistência Eventual** sem paralisar o sistema com locks distribuídos.

6 Algoritmos de Balanceamento de Carga Implementados

O Load Balancer (Porta 8000) atua como gateway de entrada com distribuição dinâmica de tráfego. Foram implementadas e testadas seis estratégias algorítmicas de balanceamento com suporte a desempate circular:

Algoritmo	Lógica Matemática e Mecanismo de Despacho	Cenário de Uso Recomendado
Round Robin (RR)	Distribuição circular determinística: nó $i = (k \bmod N)$, com alternância sequencial e equitativa.	Servidores com capacidades de processamento idênticas.
Weighted RR	Alternância baseada em peso: nós mais potentes (ex.: Servidor 01 com peso 2) atendem proporcionalmente mais chamadas.	Ambientes heterogêneos com servidores de diferentes tamanhos de CPU/RAM.
Least Connections	Despacha para o servidor com menor número de conexões ativas simultâneas: $\arg\min_i(\text{Conexões}_i)$.	Requisições com tempo de processamento heterogêneo.
Least Response Time	Prioriza o servidor com menor média móvel exponencial de latência: $\arg\min_i(\overline{\text{RTT}}_i)$.	Servidores sujeitos a oscilações térmicas ou contenção de I/O.
Patient Hash (Sticky)	Hashing consistente sobre o identificador: $i = \text{hash}(\text{paciente_id}) \bmod N$.	Preserva localidade de cache para o mesmo paciente.
Adaptive (Auto)	Mecanismo inteligente e autônomo que monitora desvios de latência e conexões ativas em voo, chaveando dinamicamente entre RR, LC e LRT sem intervenção humana.	Clusters dinâmicos sujeitos a picos imprevisíveis e degradações pontuais de nós.

Tabela 3: Comparativo analítico dos algoritmos de distribuição de carga implementados.

7 Evidências Experimentais e Validação da Suíte de Testes

Para comprovar a robustez e a integridade de todas as camadas distribuídas após a eliminação do RabbitMQ e a consolidação do middleware ESB+ORB, o projeto conta com uma abrangente suíte automatizada executada via `pytest`.

7.1 Resultados Quantitativos por Subsistema

A tabela a seguir consolida o desempenho dos testes unitários e de integração de ponta a ponta:

Subsistema / Marco Avaliado	Testes Aprovados	Taxa de Sucesso
Ingestão de Sensores e Servidores Locais (Marco 1)	24	100%
Balanceador de Carga e Algoritmos de Distribuição (Marco 2)	48	100%
Replicação Ativa Fanout e Servidor 04 Warm Standby (Marco 3)	36	100%
Middleware ESB, ORB CosNaming, Circuit Breakers e DLQ (Marco 4)	81	100%
Testes de Integridade de Comunicação e Proxies de Métricas	11	100%
Total Geral Consolidado	**200 Testes Aprovados**	**100% (0 Falhas)**

Tabela 4: Consolidação dos resultados da suíte de testes de engenharia distribuída.

7.2 Comportamento em Teste de Caos (Injeção de Falhas)

Durante os ensaios de injeção de falhas controladas:

1. Ao provocar a queda forçada do Servidor 02 (`KILL -9` na porta 8002), o Circuit Breaker registrou os erros consecutivos e abriu o circuito em menos de 150 ms.

2. O balanceador de carga removeu o nó doente do pool circular sem que o cliente produtor recebesse erros 500, alcançando 0% de perda de pacotes clínicos.

3. Ao restaurar o processo do servidor, o Circuit Breaker transitou para `HALF-OPEN`, validou o restabelecimento com sucesso e reintegrou o nó ao fluxo produtivo.

8 Conclusão

A refatoração arquitetural documentada neste relatório elevou o patamar do projeto de um protótipo acoplado a dependências externas para uma verdadeira plataforma de sistemas distribuídos de missão crítica:

- **Autonomia Total**: A eliminação do RabbitMQ livrou o sistema de dependências operacionais externas pesadas, tornando-o auto-contido, rápido e resiliente.
- **Inteligência Hospitalar no Barramento**: O middleware deixou de ser um mero conduíte de bytes e passou a agir como camada inteligente de triagem, validação e roteamento de emergências médicas via Pipeline VETRO.
- **Conformidade Rigorosa com os Marcos 3 e 4**: A persistência multi-banco com replicação fanout, o suporte a warm standby e os mecanismos autônomos de Circuit Breaker asseguram alta disponibilidade e recuperação transparente perante desastres.